

Quantifier-Free Equality and Data Structures

Almost all proofs require reasoning about equalities.

— Greg Nelson and Derek C. Oppen

Fast Decision Procedures Based on Congruence Closure, 1980

Equality is perhaps the most widely-used relation among data in programs. In this chapter, we consider equality among variables, constants, and function applications (Section 9.1); among recursive data structures (records, lists, trees, and stacks) and their elements (Section 9.4); and among elements of arrays (Section 9.5). For all three theories, we examine their quantifier-free fragments.

In the last two theories — recursive data structures and arrays — we distinguish between equality among data structures and among their elements. The quantifier-free fragment of the theory of recursive data structures can express equality among whole structures, which reduces to element-wise equality. In contrast, the quantifier-free fragment of the theory of arrays cannot express equality among entire arrays. Chapter 11 presents larger fragments of various theories of arrays that can express equality among arrays and array segments, among other properties.

Equality among variables, constants, and function applications is expressed in the theory of equality T_E . This theory is sometimes referred to as the **theory of equality with uninterpreted functions (EUF)**. Its signature contains all predicate and function symbols, yet its axioms do not interpret (assign meaning to) the symbols other than in the context of equality. Specifically, the axioms of T_E assert that a function symbol acts like a function: if two terms t_1 and t_2 are equal, then $f(t_1)$ and $f(t_2)$ are also equal. Similarly, predicate symbols act like predicates: $p(t_1)$ is equivalent to $p(t_2)$ when t_1 is equal to t_2 .

Section 9.1 presents the theory T_E and its quantifier-free fragment. Then Section 9.2 describes at a high-level the **congruence-closure algorithm** to decide T_E -satisfiability of quantifier-free Σ_E -formulae. Section 9.3 details an implementation that employs an efficient representation of Σ_E -formulae.

The congruence closure algorithm is the basis for the other decision procedures of this chapter as well. It is extended in Section 9.4 to decide satisfiability in the quantifier-free fragment of the theory of recursive data structures T_{RDS} , and in particular in the theory of lists T_{cons} . Finally, it is applied in Section 9.5 to decide satisfiability in the quantifier-free fragment of the theory of arrays T_{A} .

The quantifier-free fragment of T_{E} and its satisfiability decision procedure play a central role in combining theories that share the equality predicate. We discuss the combination of theories in Chapter 10.

9.1 Theory of Equality

Recall from Chapter 3 that the signature of T_{E} ,

$$\Sigma_{\text{E}} : \{=, a, b, c, \dots, f, g, h, \dots, p, q, r, \dots\},$$

consists of

- $=$, a binary predicate;
- and all constant, function, and predicate symbols.

As in every other theory, Σ_{E} -formulae are constructed from symbols of the signature, variables, logical connectives, and quantifiers.

The equality predicate $=$ is interpreted, or given meaning, via the axioms of T_{E} . The axioms

1. $\forall x. x = x$ (reflexivity)
2. $\forall x, y. x = y \rightarrow y = x$ (symmetry)
3. $\forall x, y, z. x = y \wedge y = z \rightarrow x = z$ (transitivity)

define $=$ to be an **equivalence relation**. These axioms give $=$ the expected meaning of equality on pairs of variable terms.

However, they do not provide the full meaning for $=$ in the context of function terms, such as in $f(x) = f(g(y, z))$. The following axiom schema stands for an infinite but countable set of axioms:

4. for each positive integer n and n -ary function symbol f ,

$$\forall \bar{x}, \bar{y}. \left(\bigwedge_{i=1}^n x_i = y_i \right) \rightarrow f(\bar{x}) = f(\bar{y}) \quad \text{(function congruence)}$$

For example, two instances of this axiom schema are the following:

$$\forall x, y. x = y \rightarrow f(x) = f(y)$$

and

$$\forall x_1, x_2, y_1, y_2. x_1 = y_1 \wedge x_2 = y_2 \rightarrow g(x_1, x_2) = g(y_1, y_2).$$

Then

$$x = g(y, z) \rightarrow f(x) = f(g(y, z))$$

is T_E -valid by the first instance. Alternately,

$$x = g(y, z) \wedge f(x) \neq f(g(y, z))$$

is T_E -unsatisfiable, where $t_1 \neq t_2$ abbreviates $\neg(t_1 = t_2)$. This axiom schema makes = a **congruence relation**.

Finally, observe that the logical operator $\leftrightarrow$ should behave on predicate formulae similarly to the way = behaves on function terms. For example, our intuition asserts that

$$x = y \rightarrow (p(x) \leftrightarrow p(y)) \quad (9.1)$$

should be T_E -valid. In Chapter 3, we list a fifth axiom schema:

5. for each positive integer n and n -ary predicate symbol p ,

$$\forall \bar{x}, \bar{y}. \left(\bigwedge_{i=1}^n x_i = y_i \right) \rightarrow (p(\bar{x}) \leftrightarrow p(\bar{y})) \quad (\text{predicate congruence})$$

Under this axiom schema, formula (9.1) is T_E -valid.

Example 9.1. The Σ_E -formula

$$f(x) = f(y) \wedge x \neq y$$

is T_E -satisfiable: a function can evaluate unequal arguments to equal values. However,

$$x = y \wedge f(x) \neq f(y)$$

is T_E -unsatisfiable: a function evaluates each value to one value, independent of representation. Here, $x = y$, so x and y represent the same value.

Finally,

$$F : f(f(f(a))) = a \wedge f(f(f(f(f(a)))) = a \wedge f(a) \neq a$$

is T_E -unsatisfiable. We can make the following intuitive argument: substituting a for $f(f(f(a)))$ in $f(f(f(f(f(a)))) = a$ by the first equality yields $f(f(a)) = a$; and substituting a for $f(f(a))$ in $f(f(f(a))) = a$ according to this new equality yields $f(a) = a$, contradicting the literal $f(a) \neq a$.

Substitution, of course, rests on the axioms of T_E : in particular, the first substitution requires four steps:

- | | |
|--------------------------------|--|
| 1. $f(f(f(f(a)))) = f(a)$ | first literal of F , (function congruence) |
| 2. $f(f(f(f(f(a)))) = f(f(a))$ | step 1, (function congruence) |
| 3. $f(f(a)) = f(f(f(f(a))))$ | step 2, (symmetry) |
| 4. $f(f(a)) = a$ | step 3, second literal of F , (transitivity) |

The decision procedure of this chapter reasons similarly. ■

The (predicate congruence) axiom schema needlessly complicates our discussion. Instead, we describe by example a simple reduction of Σ_E -formulae containing uninterpreted predicates to Σ_E -formulae without predicates other than $=$. This transformation allows us to disregard the (predicate congruence) axiom schema. For example, given Σ_E -formula

$$x = y \rightarrow (p(x) \leftrightarrow p(y)) ,$$

introduce fresh constant $\bullet$ and fresh function f_p , and write

$$x = y \rightarrow ((f_p(x) = \bullet) \leftrightarrow (f_p(y) = \bullet)) .$$

Similarly, transform

$$p(x) \wedge q(x, y) \wedge q(y, z) \rightarrow \neg q(x, z)$$

into

$$f_p(x) = \bullet \wedge f_q(x, y) = \bullet \wedge f_q(y, z) = \bullet \rightarrow f_q(x, z) \neq \bullet .$$

In the rest of this chapter, we consider Σ_E -formulae without predicates other than $=$.

T_E -satisfiability is undecidable since satisfiability in FOL is undecidable. However, satisfiability in the quantifier-free fragment of T_E is decidable. We focus on this fragment in this chapter. Therefore, when we say “ Σ_E -formula,” we mean “quantifier-free Σ_E -formula” unless otherwise stated.

Additionally, as in Chapter 8, we consider only quantifier-free Σ_E -formulae that are conjunctions of literals. Satisfiability of an arbitrary quantifier-free Σ_E -formula is considered via conversion to DNF.

9.2 Congruence Closure Algorithm

Each positive literal $s = t$ of a (conjunctive quantifier-free) Σ_E -formula F asserts an equality between two terms s and t . Applying symmetry, reflexivity, transitivity, and congruence to these equalities of F produces more equalities over terms occurring in F . Since there are only a finite number of terms in F , only a finite number of equalities among these terms are possible. Hence, one of two situations eventually occurs: either some equality is formed that directly contradicts a negative literal $s' \neq t'$ of F ; or the propagation of equalities ends without finding a contradiction. These cases correspond to T_E -unsatisfiability and T_E -satisfiability, respectively, of F .

This section develops the basic concepts for describing the algorithm mathematically as forming the **congruence closure** of the equality relation over terms asserted by F .

9.2.1 Relations

Binary Relations

Consider a set S and a binary relation R over S . For two elements $s_1, s_2 \in S$, either $s_1 R s_2$ or $\neg(s_1 R s_2)$.

The relation R is an **equivalence relation** if it is

- **reflexive**: $\forall s \in S. s R s$;
- **symmetric**: $\forall s_1, s_2 \in S. s_1 R s_2 \rightarrow s_2 R s_1$;
- **transitive**: $\forall s_1, s_2, s_3 \in S. s_1 R s_2 \wedge s_2 R s_3 \rightarrow s_1 R s_3$.

It is a **congruence relation** if it additionally obeys congruence: for every n -ary function f ,

$$\forall \bar{s}, \bar{t}. \left(\bigwedge_{i=1}^n s_i R t_i \right) \rightarrow f(\bar{s}) R f(\bar{t}) .$$

In this case, function evaluation of R -related terms yields R -related results. The way we think of equality in our everyday computations is as a congruence relation.

Classes and Partitions

Consider an equivalence relation R over a set S . The **equivalence class** of $s \in S$ under R is the set

$$[s]_R \stackrel{\text{def}}{=} \{s' \in S : s R s'\} .$$

If R is a congruence relation over S , then $[s]_R$ is the **congruence class** of s .

Example 9.2. Consider the set $\mathbb{Z}$ of integers and the equivalence relation $\equiv_2$ such that

$$m \equiv_2 n \quad \text{iff} \quad (m \bmod 2) = (n \bmod 2) .$$

$m, n \in \mathbb{Z}$ are related iff they are both even or both odd. The equivalence class of 3 under $\equiv_2$ is

$$\begin{aligned} [3]_{\equiv_2} &= \{n \in \mathbb{Z} : (n \bmod 2) = (3 \bmod 2)\} \\ &= \{n \in \mathbb{Z} : (n \bmod 2) = 1\} \\ &= \{n \in \mathbb{Z} : n \text{ is odd}\} . \end{aligned}$$

■

A **partition** P of S is a set of subsets of S that is **total**,

$$\left(\bigcup_{S' \in P} S' \right) = S ,$$

and **disjoint**,

$$\forall S_1, S_2 \in P. S_1 \neq S_2 \rightarrow S_1 \cap S_2 = \emptyset .$$

The **quotient** S/R of S by the equivalence (congruence) relation R is a partition of S : it is a set of equivalence (congruence) classes

$$S/R = \{[s]_R : s \in S\} .$$

Example 9.3. The quotient $\mathbb{Z}/\equiv_2$ is a partition: it is the set of equivalence classes

$$\{\{n \in \mathbb{Z} : n \text{ is odd}\}, \{n \in \mathbb{Z} : n \text{ is even}\}\} .$$

■

Just as an equivalence relation R induces a partition S/R of S , a given partition P of S induces an equivalence relation over S . Specifically, $s_1 R s_2$ iff for some $S' \in P$, both $s_1, s_2 \in S'$.

Relation Refinements

Consider two binary relations R_1 and R_2 over set S . R_1 is a **refinement** of R_2 , or $R_1 \prec R_2$, if

$$\forall s_1, s_2 \in S. s_1 R_1 s_2 \rightarrow s_1 R_2 s_2 .$$

We also say that R_1 **refines** R_2 . Viewing the relations as sets of pairs, $R_1 \subseteq R_2$.

Example 9.4. For $S = \{a, b\}$, $R_1 : \{a R_1 b\} \prec R_2 : \{a R_2 b, b R_2 b\}$.

Viewing the relations as sets of pairs, $R_1 \prec R_2$ iff $R_1 \subseteq R_2$.

■

Example 9.5. Consider set S , the relation

$$R_1 : \{s R_1 s : s \in S\}$$

induced by the partition

$$P_1 : \{\{s\} : s \in S\} ,$$

and the relation

$$R_2 : \{s R_2 t : s, t \in S\}$$

induced by the partition

$$P_2 : \{S\} .$$

Then $R_1 \prec R_2$.

■

Example 9.6. Consider the set $\mathbb{Z}$ and the two relations

$$R_1 : \{xR_1y : x \bmod 2 = y \bmod 2\} \text{ and } R_2 : \{xR_2y : x \bmod 4 = y \bmod 4\} .$$

Then $R_2 \prec R_1$. ■

Closures

The **equivalence closure** R^E of the binary relation R over S is the equivalence relation such that

- R refines R^E : $R \prec R^E$;
- for all other equivalence relations R' such that $R \prec R'$, either $R' = R^E$ or $R^E \prec R'$.

That is, R^E is the “smallest” equivalence relation that “covers” R .

Example 9.7. If $S = \{a, b, c, d\}$ and

$$R = \{aRb, bRc, dRd\} ,$$

then

- $aRb, bRc, dRd \in R^E$ since $R \subseteq R^E$;
- $aRa, bRb, cRc \in R^E$ by reflexivity;
- $bRa, cRb \in R^E$ by symmetry;
- $aRc \in R^E$ by transitivity;
- $cRa \in R^E$ by symmetry.

Hence,

$$R^E = \{aRb, bRa, aRa, bRb, bRc, cRb, cRc, aRc, cRa, dRd\} .$$
■

The **congruence closure** R^C of R is the “smallest” congruence relation that “covers” R . Shortly, we shall illustrate the congruence closure of a term set.

9.2.2 Congruence Closure Algorithm

Having defined several abstract concepts, we now return to the theory of equality. The **subterm set** S_F of Σ_E -formula F is the set that contains precisely the subterms of F .

Example 9.8. The subterm set of

$$F : f(a, b) = a \wedge f(f(a, b), b) \neq a$$

is

$$S_F = \{a, b, f(a, b), f(f(a, b), b)\} .$$
■

Now we relate the congruence closure of a Σ_E -formula's subterm set with its T_E -satisfiability. Given Σ_E -formula F

$$F : s_1 = t_1 \wedge \cdots \wedge s_m = t_m \wedge s_{m+1} \neq t_{m+1} \wedge \cdots \wedge s_n \neq t_n, \quad (9.2)$$

with subterm set S_F , F is T_E -satisfiable iff there exists a congruence relation $\sim$ over S_F such that

- for each $i \in \{1, \dots, m\}$, $s_i \sim t_i$;
- for each $i \in \{m+1, \dots, n\}$, $s_i \not\sim t_i$.

Such a congruence relation $\sim$ defines a T_E -interpretation $I : (D_I, \alpha_I)$ of F . D_I consists of $|S_F|/\sim$ elements, one for each congruence class of S_F under $\sim$. α_I assigns elements of D_I to the terms of S_F in a way that respects $\sim$. Finally, α_I assigns to $=$ a binary relation over D_I that behaves like $\sim$. Based on this construction, we abbreviate $(D_I, \alpha_I) \models F$ with $\sim \models F$.

The goal of the **congruence closure algorithm** is to construct the congruence relation of a formula's subterm set, or to prove that no congruence relation exists. Given formula (9.2), the algorithm performs the following steps:

1. Construct the congruence closure $\sim$ of

$$\{s_1 = t_1, \dots, s_m = t_m\}$$

over the subterm set S_F . Then

$$\sim \models s_1 = t_1 \wedge \cdots \wedge s_m = t_m.$$

2. If $s_i \sim t_i$ for any $i \in \{m+1, \dots, n\}$, return **unsatisfiable**.
3. Otherwise, $\sim \models F$, so return **satisfiable**.

How do we actually construct the congruence closure in Step 1? Initially, begin with the finest congruence relation $\sim_0$ given by the partition

$$\{\{s\} : s \in S_F\}$$

in which each term of S_F is its own congruence class. Then, for each $i \in \{1, \dots, m\}$, impose $s_i = t_i$ by merging the congruence classes

$$[s_i]_{\sim_{i-1}} \quad \text{and} \quad [t_i]_{\sim_{i-1}}$$

to form a new congruence relation $\sim_i$. To accomplish this merging, first form the union of $[s_i]_{\sim_{i-1}}$ and $[t_i]_{\sim_{i-1}}$. Then propagate any new congruences that arise within this union. In the new congruence relation $\sim_i$, $s_i \sim_i t_i$. Section 9.3 presents the details of an efficient data structure and algorithm for performing this construction.

Example 9.9. Consider the Σ_E -formula

$$F : f(a, b) = a \wedge f(f(a, b), b) \neq a.$$

Construct the following initial partition by letting each member of the subterm set S_F be its own class:

$$\{\{a\}, \{b\}, \{f(a,b)\}, \{f(f(a,b),b)\}\} .$$

According to the first literal $f(a,b) = a$, merge

$$\{f(a,b)\} \quad \text{and} \quad \{a\}$$

to form partition

$$\{\{a, f(a,b)\}, \{b\}, \{f(f(a,b),b)\}\} .$$

According to the (function congruence) axiom,

$$f(a,b) \sim a, \quad b \sim b \quad \text{implies} \quad f(f(a,b),b) \sim f(a,b) ,$$

resulting in the new partition

$$\{\{a, f(a,b), f(f(a,b),b)\}, \{b\}\} .$$

This partition represents the congruence closure of S_F . Now, is it the case that

$$\{\{a, f(a,b), f(f(a,b),b)\}, \{b\}\} \models F ?$$

No, as $f(f(a,b),b) \sim a$ but F asserts that $f(f(a,b),b) \neq a$. Hence, F is T_E -unsatisfiable. ■

Example 9.10. Consider the Σ_E -formula

$$F : f(f(f(a))) = a \wedge f(f(f(f(f(a)))) = a \wedge f(a) \neq a$$

From the subterm set S_F , the initial partition is

$$\{\{a\}, \{f(a)\}, \{f^2(a)\}, \{f^3(a)\}, \{f^4(a)\}, \{f^5(a)\}\} ,$$

where, for example, $f^3(a)$ abbreviates $f(f(f(a)))$.

According to the literal $f^3(a) = a$, merge

$$\{f^3(a)\} \quad \text{and} \quad \{a\} .$$

From the union $\{a, f^3(a)\}$, deduce the following congruence propagations:

$$f^3(a) \sim a \Rightarrow f(f^3(a)) \sim f(a) , \quad \text{i.e.,} \quad f^4(a) \sim f(a)$$

and

$$f^4(a) \sim f(a) \Rightarrow f(f^4(a)) \sim f(f(a)) , \quad \text{i.e.,} \quad f^5(a) \sim f^2(a) .$$

Thus, the final partition for this iteration is the following:

$$\{\{a, f^3(a)\}, \{f(a), f^4(a)\}, \{f^2(a), f^5(a)\}\} .$$

From the second literal, $f^5(a) = a$, merge

$$\{f^2(a), f^5(a)\} \quad \text{and} \quad \{a, f^3(a)\}$$

to form the partition

$$\{\{a, f^2(a), f^3(a), f^5(a)\}, \{f(a), f^4(a)\}\} .$$

Propagating the congruence

$$f^3(a) \sim f^2(a) \Rightarrow f(f^3(a)) \sim f(f^2(a)) , \text{ i.e., } f^4(a) \sim f^3(a)$$

yields the partition

$$\{\{a, f(a), f^2(a), f^3(a), f^4(a), f^5(a)\}\} ,$$

which represents the congruence closure in which all of S_F are equal.

Now, is it the case that

$$\{\{a, f(a), f^2(a), f^3(a), f^4(a), f^5(a)\}\} \models F ?$$

No, as $f(a) \sim a$, but F asserts that $f(a) \neq a$. Hence, F is T_E -unsatisfiable. ■

Example 9.11. Consider the Σ_E -formula

$$F : f(x) = f(y) \wedge x \neq y .$$

The subterm set S_F induces the following initial partition:

$$\{\{x\}, \{y\}, \{f(x)\}, \{f(y)\}\} .$$

Then $f(x) = f(y)$ indicates to merge

$$\{f(x)\} \quad \text{and} \quad \{f(y)\} .$$

The union $\{f(x), f(y)\}$ does not yield any new congruences, so the final partition is

$$\{\{x\}, \{y\}, \{f(x), f(y)\}\} .$$

Does

$$\{\{x\}, \{y\}, \{f(x), f(y)\}\} \models F ?$$

Yes, as $x \not\sim y$, agreeing with $x \neq y$. Hence, F is T_E -satisfiable. ■

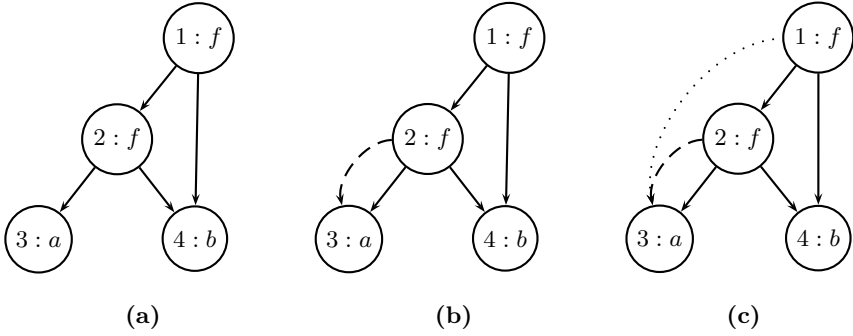


Fig. 9.1. (a) DAG representation; (b), (c) with `find`

9.3 Congruence Closure with DAGs

So far, we have considered the congruence closure algorithm at an abstract level. In this section, we describe an efficient implementation of the algorithm.

Section 9.3.1 defines a graph-based data structure for representing all members of the subterm set S_F of a Σ_E -formula F . Each node represents a subterm. Congruence classes are stored within this data structure via references between nodes. Sections 9.3.2 and 9.3.3 present algorithms that manipulate the data structure to construct the congruence closure of the relation defined by F .

9.3.1 Directed Acyclic Graphs

A **graph** $G : \langle N, E \rangle$ has a set of **nodes** $N = \{n_1, n_2, \dots, n_k\}$ and a set of **edges** $E = \{\dots, \langle n_i, n_j \rangle, \dots\}$, which consists of pairs of nodes. In a **directed graph**, edges point from one node to another. For example, the edge $\langle n_3, n_5 \rangle$ is not the same edge as $\langle n_5, n_3 \rangle$: the first points to n_5 , while the second points to n_3 . In a directed edge $\langle m, n \rangle$, m is the **source**, and n is the **target**. A **directed acyclic graph (DAG)** is a directed graph in which no subset of edges forms a directed loop, or **cycle**.

Example 9.12. Consider the directed graph

$$G : \langle N : \{1, 2, 3\}, E : \{\langle 1, 2 \rangle, \langle 2, 3 \rangle, \langle 3, 1 \rangle\} \rangle .$$

It is not a DAG because it contains the loop $1 \rightarrow 2 \rightarrow 3 \rightarrow 1$. However, the directed graph

$$G : \langle N : \{1, 2, 3\}, E : \{\langle 1, 2 \rangle, \langle 2, 3 \rangle, \langle 1, 3 \rangle\} \rangle$$

is a DAG. ■

The congruence closure algorithm uses a DAG to represent terms of the subterm set. The DAG of Figure 9.1(a) represents the subterm set

$$\{a, b, f(a, b), f(f(a, b), b)\}.$$

Each node has a unique number identifying it and a text label representing the root constant or function symbol of the associated term. Edges point from a function symbol to its arguments. For example, node 3 represents the term a ; node 2 represents the term $f(a, b)$; and node 1 represents the term $f(f(a, b), b)$. Node 4, representing the term b , is shared by the terms $f(a, b)$ and $f(f(a, b), b)$.

So far, the following data type is sufficient to represent subterms:

```
type node = {
  id      : id
  fn      : string
  args    : id list
}
```

The **id** field holds the node's unique identification number; the **fn** field holds the constant or function symbol; and the **args** field holds a list of identification numbers representing the function arguments. For example, node 2 of Figure 9.1(a) has **id** = 2, **fn** = f , and **args** = [3; 4]; and node 3 has **id** = 3, **fn** = a , and **args** = [].

To represent congruence classes, we augment the data type with one more field:

```
type node = {
  id          : id
  fn          : string
  args        : id list
  mutable find : id
}
```

The **mutable** keyword indicates that the value of the field can be modified. The **find** field holds the identification number of another node (possibly itself) in its congruence class. Following a chain of **find** references leads to the **representative** of the congruence class. A representative node's **find** field points to the node itself.

Consider the DAG of Figure 9.1(b). The dashed edge from node 2 to node 3 indicates that node 2's **find** field points to node 3, and thus that nodes 2 and 3 are in the same congruence class

$$\{f(a, b), a\}.$$

To avoid cluttering the illustrations, we do not draw dashed lines from a node to itself when its **find** is set to itself. Thus, the DAG of Figure 9.1(b) represents the partition

$$\{\{f(a, b), a\}, \{b\}, \{f(f(a, b), b)\}\};$$

and, for example, the `find` field of node 1 is 1. Also, since the `find` of node 3 is 3, node 3 represents the congruence class of nodes 2 and 3.

In Figure 9.1(c), the dashed and dotted edges play the same role for now, each representing the value of the source node's `find` field. The DAG represents the partition

$$\{\{f(f(a, b), b), f(a, b), a\}, \{b\}\}.$$

Node 3 is the (unique) representative of its congruence class. We shall see that the algorithm maintains exactly one representative for each congruence class.

Finally, merging congruence classes requires accessing parent terms (*e.g.*, $f(a, b)$ is the parent of a and b). Thus, we add a final mutable field:

```
type node = {
  id          : id
  fn          : string
  args        : id list
  mutable find : id
  mutable ccpair : id set
}
```

If a node is the representative for its congruence class, then its `ccpair` (for **congruence closure parents**) field stores the set of all parents of all nodes in its congruence class. A non-representative node's `ccpair` field is empty. In Figure 9.1(b), the `ccpair`

- of 1 is $\emptyset$ because 1 is its own representative, but it lacks parents;
- of 2 is $\emptyset$ because 2 is not a representative;
- of 3 is $\{1, 2\}$ because 3 represents the class $\{2, 3\}$ in which the parent of 2 is 1 and the parent of 3 is 2;
- of 4 is $\{1, 2\}$ because 4 represents itself and has parents $\{1, 2\}$.

The DAG representation of node 2 is thus

```
{ id    = 2;
  fn    = f;
  args  = [3; 4];
  find  = 3;
  ccpair =  $\emptyset$ ;
}
```

Similarly, the DAG representation of node 3 is

```
{ id    = 3;
  fn    = a;
  args  = [];
  find  = 3;
  ccpair = {1, 2};
}
```

9.3.2 Basic Operations

In this section, we define the **union-find** algorithm on the DAG data structure. Generally, a union-find algorithm provides an efficient means of manipulating sets. It represents each set by a single representative element. Calling the **FIND** function on any element returns its set's unique representative. Calling the **UNION** function on two elements unions the two elements' sets and fixes a unique representative for the union set.

The union-find algorithm on DAGs is sufficient for manipulating equivalence classes of terms. In Section 9.3.3, we extend the union-find algorithm to manipulate congruence classes.

NODE

NODE i returns the node n with **id** i . Thus, $(\text{NODE } i).\text{id} = i$. For example, in Figure 9.1(b), $(\text{NODE } 2).\text{find} = 3$.

FIND

The **FIND** function returns the representative of a node's equivalence class. It follows **find** edges until it finds a self-loop:

```
let rec FIND  $i$  =
  let  $n$  = NODE  $i$  in
  if  $n.\text{find} = i$  then  $i$  else FIND  $n.\text{find}$ 
```

Example 9.13. In the DAG of Figure 9.1(b), **FIND** 2 is 3. **FIND** follows the **find** edge of 2 to 3; then it recognizes the self-loop and thus returns 3. ■

UNION

The **UNION** function returns the union of two equivalence classes, given two node identities i_1 and i_2 . It first finds the representatives n_1 and n_2 of i_1 's and i_2 's equivalence classes, respectively. Next, it sets n_1 's **find** to n_2 's representative, which is the identity of n_2 itself. Now n_2 represents the new larger equivalence class.

Finally, it combines the congruence closure parents, storing the new set in n_2 's **ccpar** field because n_2 is the representative of the union equivalence class. This last step is not strictly part of the union-find algorithm (which, recall, computes equivalence classes); rather, it is intended for when we use the union-find algorithm to compute congruence classes. In code,

```
let UNION  $i_1$   $i_2$  =
  let  $n_1$  = NODE (FIND  $i_1$ ) in
  let  $n_2$  = NODE (FIND  $i_2$ ) in
   $n_1.\text{find} \leftarrow n_2.\text{find};$ 
   $n_2.\text{ccpar} \leftarrow n_1.\text{ccpar} \cup n_2.\text{ccpar};$ 
   $n_1.\text{ccpar} \leftarrow \emptyset$ 
```

Example 9.14. Consider the DAG of Figure 9.1(b). To compute `UNION 1 2`, `UNION` finds $n_1 = 1$ and $n_2 = 3$ as the representatives of $i_1 = 1$ and $i_2 = 2$, respectively. It sets 1's `find` field to 3's `find`, which is 3, and combines the `ccpars`, setting 3's `ccpar` to $\{1, 2\}$ and 1's `ccpar` to $\emptyset$. The result is Figure 9.1(c). For now, ignore the difference between dotted and dashed lines. ■

CCPAR

The simple function `CCPAR  $i$` returns the parents of all nodes in i 's congruence class:

```
let CCPAR  $i$  =
  (NODE (FIND  $i$ )).ccpar
```

9.3.3 Congruence Closure Algorithm

We are ready to build the congruence closure algorithm using the basic operations. First we define the `CONGRUENT` function to check whether two nodes that are not in the same congruence class are in fact congruent. Then we define the `MERGE` function to merge two congruence classes and to propagate the effects of new congruences recursively.

CONGRUENT

`CONGRUENT  $i_1$   $i_2$` tests whether i_1 and i_2 are congruent. Let n_1 and n_2 be the nodes with identities i_1 and i_2 , respectively. If their `fn` fields or their numbers of arguments are different, then they cannot be congruent, so `CONGRUENT` returns `false`. Otherwise, if any argument of n_1 is not in the congruence class of the corresponding argument of n_2 , then the terms are not congruent, so `CONGRUENT` returns `false`. If both of these tests are passed, then n_1 and n_2 are congruent, so `CONGRUENT` returns `true`. Formally,

```
let CONGRUENT  $i_1$   $i_2$  =
  let  $n_1$  = NODE  $i_1$  in
  let  $n_2$  = NODE  $i_2$  in
   $n_1$ .fn =  $n_2$ .fn
   $\wedge$   $|n_1$ .args| =  $|n_2$ .args|
   $\wedge$   $\forall i \in \{1, \dots, |n_1$ .args|\}. FIND  $n_1$ .args[ $i$ ] = FIND  $n_2$ .args[ $i$ ]
```

Example 9.15. Consider the DAG of Figure 9.1(b). Are nodes 1 and 2 congruent? `CONGRUENT` notes that

- their `fn` fields are both f : n_1 .fn = n_2 .fn = f ;
- their numbers of arguments are both 2;
- their left arguments $f(a, b)$ and a are both congruent to 3:
 n_1 .args = $[2; 4]$, n_2 .args = $[3; 4]$, and `FIND 2` = `FIND 3` = 3;

- and their right arguments b and b are both congruent to 4:
 $n_1.\mathbf{args} = [2; 4]$, $n_2.\mathbf{args} = [3; 4]$, and $\mathbf{FIND} \ 4 = \mathbf{FIND} \ 4 = 4$.

Therefore, nodes 1 and 2 are congruent. ■

MERGE

Next, $\mathbf{MERGE} \ i_1 \ i_2$ merges the congruence classes of i_1 and i_2 . If $\mathbf{FIND} \ i_1 = \mathbf{FIND} \ i_2$, then the two nodes are already in the same congruence class, so $\mathbf{MERGE}$ returns without changing the DAG structure. Otherwise, let P_{i_1} and P_{i_2} store the current values of $\mathbf{CCPAR} \ i_1$ and $\mathbf{CCPAR} \ i_2$, respectively. $\mathbf{MERGE}$ first calls $\mathbf{UNION} \ i_1 \ i_2$ to compute the merged equivalence class. Then, for each pair of terms $t_1, t_2 \in P_{i_1} \times P_{i_2}$, it recursively calls $\mathbf{MERGE} \ t_1 \ t_2$ if $\mathbf{CONGRUENT} \ t_1 \ t_2$. This last step propagates the effects of new congruences according to the (**function congruence**) axiom schema. Upon completion, the new partition represents a congruence relation in which i_1 and i_2 are congruent. In code,

```

let rec  $\mathbf{MERGE} \ i_1 \ i_2 =$ 
  if  $\mathbf{FIND} \ i_1 \neq \mathbf{FIND} \ i_2$  then begin
    let  $P_{i_1} = \mathbf{CCPAR} \ i_1$  in
    let  $P_{i_2} = \mathbf{CCPAR} \ i_2$  in
     $\mathbf{UNION} \ i_1 \ i_2$ ;
    foreach  $t_1, t_2 \in P_{i_1} \times P_{i_2}$  do
      if  $\mathbf{FIND} \ t_1 \neq \mathbf{FIND} \ t_2 \ \wedge \ \mathbf{CONGRUENT} \ t_1 \ t_2$ 
      then  $\mathbf{MERGE} \ t_1 \ t_2$ 
    done
  end

```

9.3.4 Decision Procedure for T_E -Satisfiability

Given Σ_E -formula

$$F : s_1 = t_1 \ \wedge \ \cdots \ \wedge \ s_m = t_m \ \wedge \ s_{m+1} \neq t_{m+1} \ \wedge \ \cdots \ \wedge \ s_n \neq t_n$$

with subterm set S_F , perform the following steps:

1. Construct the initial DAG for the subterm set S_F .
2. For $i \in \{1, \dots, m\}$, $\mathbf{MERGE} \ s_i \ t_i$.
3. If $\mathbf{FIND} \ s_i = \mathbf{FIND} \ t_i$ for some $i \in \{m+1, \dots, n\}$, return **unsatisfiable**.
4. Otherwise (if $\mathbf{FIND} \ s_i \neq \mathbf{FIND} \ t_i$ for all $i \in \{m+1, \dots, n\}$) return **satisfiable**.

Example 9.16. Consider the Σ_E -formula

$$F : f(a, b) = a \ \wedge \ f(f(a, b), b) \neq a .$$

The subterm set is

$$S_F = \{a, b, f(a, b), f(f(a, b), b)\} ,$$

resulting in the initial partition

$$\{\{a\}, \{b\}, \{f(a, b)\}, \{f(f(a, b), b)\}\}$$

in which each term is its own congruence class. The DAG of Figure 9.1(a) represents this initial partition.

According to the literal $f(a, b) = a$, MERGE 2 3. FIND 2 $\neq$ FIND 3, so let

$$P_2 = \text{CCPAR } 2 = \{1\} \quad \text{and} \quad P_3 = \text{CCPAR } 3 = \{2\} .$$

The UNION 2 3 computation results in the DAG of Figure 9.1(b) in which 2 and 3 are equivalent. This new equivalence makes 1 and 2 congruent, so MERGE 1 2 is called recursively.

In computing MERGE 1 2,

$$P_1 = \text{CCPAR } 1 = \emptyset \quad \text{and} \quad P_2 = \text{CCPAR } 2 = \{1, 2\} ,$$

so $P_1 \times P_2 = \emptyset$. Hence, after computing UNION 1 2, the computation finishes with the DAG of Figure 9.1(c). The dotted edge distinguishes the deduced merge from the merge dictated by the equality in F , marked by the dashed edge. This final DAG represents the partition

$$\{\{a, f(a, b), f(f(a, b), b)\}, \{b\}\}$$

and thus the congruence relation in which a , $f(a, b)$, and $f(f(a, b), b)$ are congruent. Does

$$\{\{a, f(a, b), f(f(a, b), b)\}, \{b\}\} \models F ?$$

No, as $f(f(a, b), b) \sim a$, but F asserts that $f(f(a, b), b) \neq a$. Hence, F is T_E -unsatisfiable. ■

Example 9.17. Consider the Σ_E -formula

$$F : f(f(f(a))) = a \wedge f(f(f(f(f(a)))) = a \wedge f(a) \neq a ,$$

which induces the initial partition and DAG shown in Figure 9.2(a). According to the literal $f(f(f(a))) = a$, MERGE 3 0. On this initial merge

$$P_3 = \{4\} \quad \text{and} \quad P_0 = \{1\} .$$

Additionally, the labels of both 4 and 1 are f , and their arguments, 3 and 0 respectively, are congruent after UNION 3 0. Thus, recursively MERGE 4 1. After UNION 4 1, their parents 5 and 2 are congruent, so MERGE 5 2. The recursion finishes after UNION 5 2 since $P_5 = \emptyset$, resulting in the DAG of Figure 9.2(b).

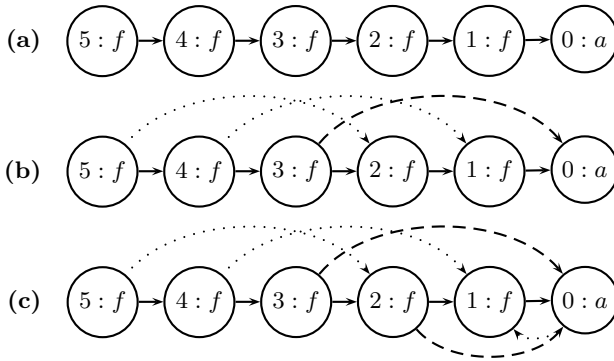


Fig. 9.2. DAGs for Example 9.17

The dotted edges distinguish deduced merges from merges dictated by F , which are marked by dashed edges. Thus, the partition is now

$$\{\{a, f^3(a)\}, \{f(a), f^4(a)\}, \{f^2(a), f^5(a)\}\} .$$

Next, according to the literal $f(f(f(f(f(a)))))) = a$, MERGE 5 0. FIND 5 = 2 and FIND 0 = 0, so

$$P_5 = \{3\} \quad \text{and} \quad P_0 = \{1, 4\} .$$

After completing UNION 5 0 (by adding the dashed line from 2 to 0 in Figure 9.2(c)), it is the case that CONGRUENT 3 1, so MERGE 3 1. This merge causes the final UNION 3 1, resulting in the dotted line from 0 to 1 in Figure 9.2(c). Figure 9.2(c) represents the partition

$$\{\{a, f(a), f^2(a), f^3(a), f^4(a), f^5(a)\}\} .$$

Now, does

$$\{\{a, f(a), f^2(a), f^3(a), f^4(a), f^5(a)\}\} \models F ?$$

No, as $f(a) \sim a$, but F asserts that $f(a) \neq a$. Hence, F is T_E -unsatisfiable. ■

Theorem 9.18 (Sound & Complete). *Quantifier-free conjunctive Σ_E -formula F is T_E -satisfiable iff the congruence closure algorithm returns satisfiable.*

9.3.5 ★Complexity

Let e be the number of edges and n be the number of nodes in the initial DAG.

Theorem 9.19 (Complexity). *The congruence closure algorithm runs in time $O(e^2)$ for $O(n)$ MERGES.*

However, Downey, Sethi, and Tarjan described an algorithm with $O(e \log e)$ average running time for $O(n)$ MERGES. Computing T_E -satisfiability is inexpensive.

9.4 Recursive Data Structures

Recursive data structures include records, lists, trees, stacks, and queues. The theory T_{RDS} can model records, lists, trees, and stacks, but not queues: whereas a particular list has a single representation, queues — in which only order matters — do not. In this section, we discuss the theory of lists T_{cons} for ease of exposition. Both its axiomatization and the decision procedure for the quantifier-free fragment rely on our discussion of T_E .

Recall that the signature of T_{cons} is

$$\Sigma_{\text{cons}} : \{\text{cons}, \text{car}, \text{cdr}, \text{atom}, =\},$$

where

- **cons** is a binary function, called the **constructor**; $\text{cons}(a, b)$ represents the list constructed by prepending a to b ;
- **car** is a unary function, called the **left projector**: $\text{car}(\text{cons}(a, b)) = a$;
- **cdr** is a unary function, called the **right projector**: $\text{cdr}(\text{cons}(a, b)) = b$;
- **atom** is a unary predicate;
- **=** is a binary predicate.

Its axioms are the following:

1. the axioms of (reflexivity), (symmetry), and (transitivity) of T_E
2. instantiations of the (function congruence) axiom schema for **cons**, **car**, and **cdr**:

$$\forall x_1, x_2, y_1, y_2. x_1 = x_2 \wedge y_1 = y_2 \rightarrow \text{cons}(x_1, y_1) = \text{cons}(x_2, y_2)$$

$$\forall x, y. x = y \rightarrow \text{car}(x) = \text{car}(y)$$

$$\forall x, y. x = y \rightarrow \text{cdr}(x) = \text{cdr}(y)$$

3. an instantiation of the (predicate congruence) axiom schema for **atom**:

$$\forall x, y. x = y \rightarrow (\text{atom}(x) \leftrightarrow \text{atom}(y))$$

4. $\forall x, y. \text{car}(\text{cons}(x, y)) = x$ (left projection)
5. $\forall x, y. \text{cdr}(\text{cons}(x, y)) = y$ (right projection)
6. $\forall x. \neg \text{atom}(x) \rightarrow \text{cons}(\text{car}(x), \text{cdr}(x)) = x$ (construction)
7. $\forall x, y. \neg \text{atom}(\text{cons}(x, y))$ (atom)

As in our discussion of T_E , we consider only quantifier-free conjunctive Σ_{cons} -formulae. Again, satisfiability of quantifier-free but non-conjunctive Σ_{cons} -formulae can be decided by converting to DNF and checking each disjunct. However, the decision procedure does not extend to the quantified case since satisfiability in the full theory T_{cons} is undecidable.

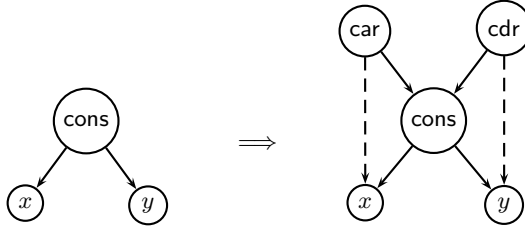


Fig. 9.3. The transformation in Step 2 of the decision procedure

Given a quantifier-free conjunctive Σ_{cons} -formula F , $\neg\text{atom}(u_i)$ literals are removed in a preprocessing step that follows from the (construction) axiom. Replace

$$\neg\text{atom}(u_i) \quad \text{with} \quad u_i = \text{cons}(u_i^1, u_i^2) .$$

Now consider Σ_{cons} -formula

$$F : s_1 = t_1 \wedge \cdots \wedge s_m = t_m \wedge s_{m+1} \neq t_{m+1} \wedge \cdots \wedge s_n \neq t_n \\ \wedge \text{atom}(u_1) \wedge \cdots \wedge \text{atom}(u_\ell)$$

in which s_i , t_i , and u_i are T_{cons} -terms. To decide its T_{cons} -satisfiability, perform the following steps:

1. Construct the initial DAG for the subterm set S_F .
2. For each node n such that $n.\text{fn} = \text{cons}$,
 - add $\text{car}(n)$ to the DAG and MERGE $\text{car}(n)$ $n.\text{args}[1]$;
 - add $\text{cdr}(n)$ to the DAG and MERGE $\text{cdr}(n)$ $n.\text{args}[2]$
 by the (left projection) and (right projection) axioms. See Figure 9.3.
3. For $i \in \{1, \dots, m\}$, MERGE s_i t_i .
4. For $i \in \{m+1, \dots, n\}$, if FIND $s_i =$ FIND t_i , return unsatisfiable.
5. For $i \in \{1, \dots, \ell\}$ if $\exists v.$ FIND $v =$ FIND $u_i \wedge v.\text{fn} = \text{cons}$, return unsatisfiable by axiom (atom).
6. Otherwise, return satisfiable.

Steps 1, 3, 4, and 6 are identical to Steps 1-4 of the decision procedure for T_E . Because of their similarity, it is simple to combine the two theories.

Example 9.20. Consider the $(\Sigma_{\text{cons}} \cup \Sigma_E)$ -formula

$$F : \text{car}(x) = \text{car}(y) \wedge \text{cdr}(x) = \text{cdr}(y) \wedge f(x) \neq f(y) \\ \wedge \neg\text{atom}(x) \wedge \neg\text{atom}(y)$$

in which the function symbol f is in $\Sigma_ =$. Is it $(T_{\text{cons}} \cup T_E)$ -satisfiable? According to the final two literals, x and y are non-atom structures; thus, the first two literals imply that $x = y$. Yet the third literal, $f(x) \neq f(y)$, contradicts this

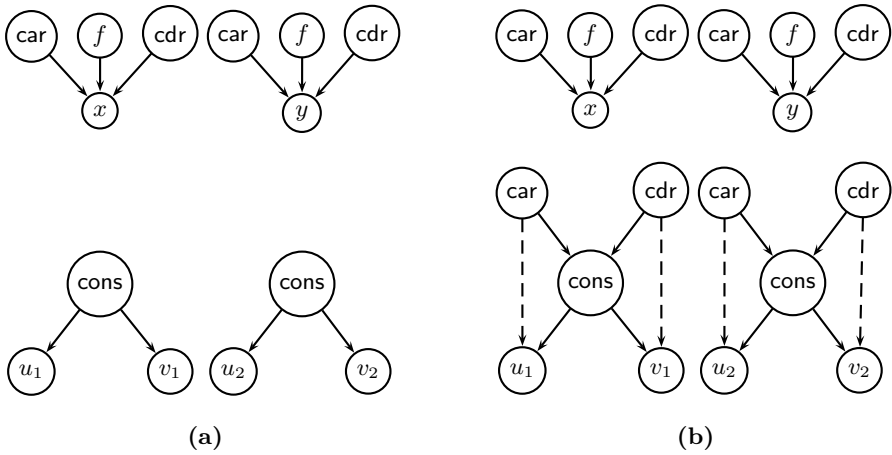


Fig. 9.4. DAG after (a) Step 1 and (b) Step 2

equality according to the (**congruence**) axiom of T_E . Hence, F is $(T_{\text{cons}} \cup T_E)$ -unsatisfiable.

To prepare F for the decision procedure, rewrite F according to the (**construction**) axiom:

$$F' : \text{car}(x) = \text{car}(y) \wedge \text{cdr}(x) = \text{cdr}(y) \wedge f(x) \neq f(y) \\ \wedge x = \text{cons}(u_1, v_1) \wedge y = \text{cons}(u_2, v_2) .$$

The first two and final two literals imply that $u_1 = u_2$ and $v_1 = v_2$ so that again $x = y$. The remaining reasoning is as for F .

Let us apply the decision procedure to F' . The initial DAG of F' is displayed in Figure 9.4(a). Figure 9.4(b) displays the DAG after Step 2.

According to the literals $\text{car}(x) = \text{car}(y)$ and $\text{cdr}(x) = \text{cdr}(y)$, compute

$$\text{MERGE } \text{car}(x) \text{ car}(y) \quad \text{and} \quad \text{MERGE } \text{cdr}(x) \text{ cdr}(y) ,$$

which add the two dashed arrows on the top of Figure 9.5(a). Then according to literal $x = \text{cons}(u_1, v_1)$,

$$\text{MERGE } x \text{ cons}(u_1, v_1) ,$$

which adds the dashed arrow from x to cons in Figure 9.5(a). Consequently, $\text{car}(x)$ and $\text{car}(\text{cons}(u_1, v_1))$ become congruent. Since

$$\text{FIND } \text{car}(x) = \text{car}(y) \quad \text{and} \quad \text{FIND } \text{car}(\text{cons}(u_1, v_1)) = u_1 ,$$

the **find** of $\text{car}(y)$ is set to point to u_1 during the subsequent **UNION**, resulting in the left dotted arrow of Figure 9.5(a). Similarly, $\text{cdr}(x)$ and $\text{cdr}(\text{cons}(u_1, v_1))$ become congruent, with similar effects (the right dotted arrow of Figure

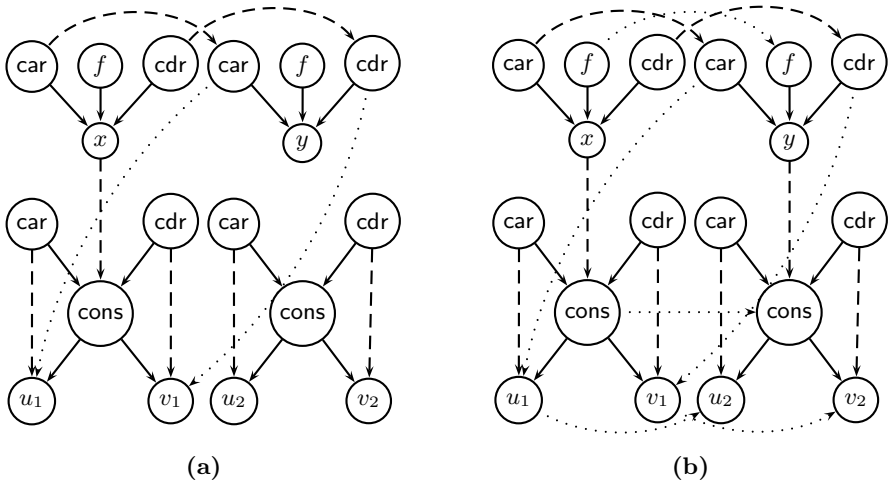


Fig. 9.5. (a) Intermediate DAG and (b) final DAG

9.5(a). The state of the DAG after these MERGES is shown in Figure 9.5(a). Dashed lines indicate merges that arise directly from the literals of F' ; dotted lines indicate deduced merges.

Next, according to the literal $y = \text{cons}(u_2, v_2)$,

MERGE $y \text{ cons}(u_2, v_2)$,

resulting in the new dashed line from y to $\text{cons}(u_2, v_2)$ in Figure 9.5(b). This merge produces two new congruences:

$$\text{car}(y) = \text{car}(\text{cons}(u_2, v_2)) \quad \text{and} \quad \text{cdr}(y) = \text{cdr}(\text{cons}(u_2, v_2)) .$$

Trace through the actions of these MERGES to understand the addition of the two bottom dotted arrows from u_1 to u_2 and from v_1 to v_2 in Figure 9.5(b). During the computation

MERGE $\text{cdr}(y) \text{ cdr}(\text{cons}(u_2, v_2))$,

$\text{cons}(u_1, v_1)$ and $\text{cons}(u_2, v_2)$ become congruent; then **find** of $\text{cons}(u_1, v_1)$ is set to point to $\text{cons}(u_2, v_2)$, as represented by the new horizontal dashed line in Figure 9.5(b). This congruence causes the additional congruence

$$f(x) = f(y) ;$$

MERGE $f(x) f(y)$ produces the final dotted edge from $f(x)$ to $f(y)$. Figure 9.5(b) displays the final DAG.

Does this DAG model F ? No, as **FIND** $f(x)$ is $f(y)$ and **FIND** $f(y)$ is $f(y)$ so that $f(x) \sim f(y)$; however, F asserts that $f(x) \neq f(y)$. F is thus $(T_{\text{cons}} \cup T_E)$ -unsatisfiable. ■

9.5 Arrays

Arrays are a basic nonrecursive data type in programming languages, so reasoning about them is important. In this section, we present a decision procedure for the quantifier-free fragment of T_A . This fragment is not expressive: one can assert properties only of individual elements, not of entire arrays. Chapter 11 examines a decision procedure for a more expressive fragment.

Recall from Chapter 3 that the theory of arrays T_A has signature

$$\Sigma_A : \{ \cdot[\cdot], \cdot\langle \cdot \triangleleft \cdot \rangle, = \} ,$$

where

- $a[i]$ is a binary function: $a[i]$ represents the value of array a at position i ;
- $a\langle i \triangleleft v \rangle$ is a ternary function: $a\langle i \triangleleft v \rangle$ represents the modified array a in which position i has value v ;
- and $=$ is a binary predicate.

The axioms of T_A are the following:

1. the axioms of (reflexivity), (symmetry), and (transitivity) of T_E
2. $\forall a, i, j. i = j \rightarrow a[i] = a[j]$ (array congruence)
3. $\forall a, v, i, j. i = j \rightarrow a\langle i \triangleleft v \rangle[j] = v$ (read-over-write 1)
4. $\forall a, v, i, j. i \neq j \rightarrow a\langle i \triangleleft v \rangle[j] = a[j]$ (read-over-write 2)

We consider T_A -satisfiability in the quantifier-free fragment of T_A . As usual, we consider only conjunctive Σ_A -formulae since conversion to DNF extends the decision procedure to arbitrary quantifier-free Σ_A -formulae.

The decision procedure for T_A -satisfiability of quantifier-free Σ_A -formula F is based on a reduction to T_E -satisfiability via applications of the (read-over-write) axioms. Intuitively, if F does not contain any write terms, then the read terms can be viewed as uninterpreted function terms. Otherwise, any write term must occur in the context of a read — as a read-over-write term $a\langle i \triangleleft v \rangle[j]$ — since arrays themselves cannot be asserted to be equal or not equal. In this case, the (read-over-write) axioms can be applied to deconstruct the read-over-write terms. In detail, to decide the T_A -satisfiability of F , perform the following recursive steps.

Step 1

If F does not contain any write terms $a\langle i \triangleleft v \rangle$, perform the following steps:

1. Associate each array variable a with a fresh function symbol f_a , and replace each read term $a[i]$ with $f_a(i)$.
2. Decide and return the T_E -satisfiability of the resulting formula.

Step 2

Select some read-over-write term $a\langle i \triangleleft v \rangle[j]$ (recall that a may itself be a write term), and split on two cases:

1. According to (read-over-write 1), replace

$$F[a\langle i \triangleleft v \rangle[j]] \quad \text{with} \quad F_1 : F[v] \wedge i = j ,$$

and recurse on F_1 . If F_1 is found to be T_A -satisfiable, return **satisfiable**.

2. According to (read-over-write 2), replace

$$F[a\langle i \triangleleft v \rangle[j]] \quad \text{with} \quad F_2 : F[a[j]] \wedge i \neq j ,$$

and recurse on F_2 . If F_2 is found to be T_A -satisfiable, return **satisfiable**.

If both F_1 and F_2 are found to be T_A -unsatisfiable, return **unsatisfiable**.

Example 9.21. Consider Σ_A -formula

$$F : i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge a\langle i_1 \triangleleft v_1 \rangle\langle i_2 \triangleleft v_2 \rangle[j] \neq a[j] .$$

Recall that $a\langle i_1 \triangleleft v_1 \rangle\langle i_2 \triangleleft v_2 \rangle[j]$ abbreviates the term

$$((a\langle i_1 \triangleleft v_1 \rangle)\langle i_2 \triangleleft v_2 \rangle)[j] .$$

F contains a write term, so select a read-over-write term to deconstruct:

$$a\langle i_1 \triangleleft v_1 \rangle\langle i_2 \triangleleft v_2 \rangle[j] .$$

According to (read-over-write 1), assume $i_2 = j$ and recurse on

$$F_1 : i_2 = j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge v_2 \neq a[j] .$$

F_1 does not contain any write terms, so rewrite it to

$$F'_1 : i_2 = j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge f_a(j) = v_1 \wedge v_2 \neq f_a(j) .$$

The first two literals imply that $i_1 = i_2$, contradicting the third literal, so F'_1 is T_E -unsatisfiable.

Returning, we try the second case: according to (read-over-write 2), assume $i_2 \neq j$ and recurse on

$$F_2 : i_2 \neq j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge a\langle i_1 \triangleleft v_1 \rangle[j] \neq a[j] .$$

F_2 contains a write term, so select the only read-over-write term to deconstruct. According to (read-over-write 1), assume $i_1 = j$ and recurse on

$$F_3 : i_1 = j \wedge i_2 \neq j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge v_1 \neq a[j] .$$

Clearly, following this path leads to a contradiction because of the final two terms. Thus, according to (read-over-write 2), assume $i_1 \neq j$ and recurse on

$$F_4 : i_1 \neq j \wedge i_2 \neq j \wedge i_1 = j \wedge i_1 \neq i_2 \wedge a[j] = v_1 \wedge a[j] \neq a[j] .$$

But the final literal is contradictory. As all branches have been tried, F is T_A -unsatisfiable. ■

Theorem 9.22 (Sound & Complete). *Given quantifier-free conjunctive Σ_A -formula F , the decision procedure returns satisfiable iff F is T_A -satisfiable; otherwise, it returns unsatisfiable.*

Theorem 9.23 (Complexity). *T_A -satisfiability of quantifier-free conjunctive Σ_A -formulae is NP-complete.*

The power of disjunction manifested in Steps 2(a) and 2(b) of the decision procedure results in this complexity.

Proof. That the problem is in NP is simple: for a given formula F , guess a formula like F , except that for every instance of a read-over-write term $a\langle i \triangleleft v \rangle[j]$, it includes as a conjunction either $i = j$ or $i \neq j$. Only a linear number of literals are added. Then check T_A -satisfiability of this formula, using the new literals to simplify read-over-write terms.

To prove NP-hardness, we reduce from satisfiability of PL formulae, which is NP-complete. Given a propositional formula F in CNF, assert

$$v_P \neq v_{\neg P}$$

for each variable P in F ; v_P and $v_{\neg P}$ represent the values of P and $\neg P$, respectively. Introduce a fresh constant $\bullet$. Then consider the n th clause of F , say

$$(\neg P \vee Q \vee \neg R) .$$

In this case, assert

$$a[j_n] \neq \bullet \wedge a\langle i_P \triangleleft v_{\neg P} \rangle \langle i_Q \triangleleft v_Q \rangle \langle i_R \triangleleft v_{\neg R} \rangle [j_n] = \bullet .$$

j_n must be equal to one of the introduced values at i_P , i_Q , or i_R . Therefore, at cell j_n the corresponding value ($v_{\neg P}$, v_Q , or $v_{\neg R}$) must equal $\bullet$. In this fashion, add an assertion for each clause of F . Conjoin all assertions to form quantifier-free conjunctive Σ_A -formula G . G is equisatisfiable to F : $v_P = \bullet$ iff P is true; and $v_{\neg P}$ iff P is false. G is also of size polynomial in the size of F . Thus, deciding T_A -satisfiability of G decides propositional satisfiability of F , so T_A -satisfiability is NP-complete. ■

9.6 Summary

This chapter presents the congruence closure algorithm and applications to deciding satisfiability in the quantifier-free fragments of T_E , T_{cons} , and T_A . It covers:

- The *congruence closure algorithm* at an abstract level. Relations, equivalence relations, congruence relations. Partitions; equivalence and congruence classes. Closures.

- The *DAG-based implementation*. Directed acyclic graph representation of formulae. The union-find algorithm. Merging.
- *Recursive data structures*. These structures include records, lists, stacks, and trees, but not queues. The decision procedure extends the congruence closure algorithm.
- *Arrays*. The decision procedure branches based on read-over-write terms according to the read-over-write axioms. When all write terms have been removed on one branch, the congruence closure algorithm is applied.

Equality is found within many first-order theories, including all the theories studied in this book. Additionally, equality and the congruence closure algorithm are the unifying components of the Nelson-Oppen combination method studied in Chapter 10. Hence, reasoning about it is fundamental. Fortunately, satisfiability is efficiently decidable using the congruence closure algorithm.

Uninterpreted functions are used to represent data structures in the context of T_{RDS} and T_A . In applications such as software and hardware verification, they allow abstracting away implementations of select components to simplify reasoning.

The DAG-based data structure that is the basis of the congruence closure algorithm is used whenever subformulae must be represented uniquely, either for algorithm correctness or for space and time considerations. Exercises 9.4 and 9.5 explore this data structure.

So far, we have seen three types of decision procedures. The quantifier-elimination procedures of Chapter 7 manipulate formulae to construct equivalent quantifier-free (and possibly variable-free) formulae. The simplex method of Chapter 8 works explicitly with the underlying structure of the set of satisfying interpretations. The congruence closure algorithm shares characteristics of both types of procedures: it manipulates formulae efficiently using the DAG-based algorithm; but it also represents satisfying interpretations explicitly as congruence classes of terms via the `find` pointers.

Bibliographic Remarks

The quantifier-free fragment of T_E was first proved decidable by Ackermann in 1954 [1] and later studied by various teams in the late 1970s. Shostak [83], Nelson and Oppen [66], and Downey, Sethi, and Tarjan [29] present alternate solutions to the problem. We discuss the method of Nelson and Oppen [66].

Oppen presents a theory of acyclic recursive data structures [69, 71]. The decision problem in the quantifier-free fragment of this theory is decidable in linear time, and the full theory is decidable. Our presentation is based on the work of Nelson and Oppen [66] for possibly-cyclic data structures.

McCarthy proposes the axiomatization of arrays based on read-over-write [58]. James King implemented the decision procedure for the quantifier-free fragment as part of his thesis work [50].

Exercises

9.1 (DP for T_E). Apply the decision procedure for T_E to the following Σ_E -formulae. Provide a level of detail as in Example 9.10.

- (a) $f(x, y) = f(y, x) \wedge f(a, y) \neq f(y, a)$
- (b) $f(g(x)) = g(f(x)) \wedge f(g(f(y))) = x \wedge f(y) = x \wedge g(f(x)) \neq x$
- (c) $f(f(f(a))) = f(f(a)) \wedge f(f(f(f(a)))) = a \wedge f(a) \neq a$
- (d) $f(f(f(a))) = f(a) \wedge f(f(a)) = a \wedge f(a) \neq a$
- (e) $p(x) \wedge f(f(x)) = x \wedge f(f(f(x))) = x \wedge \neg p(f(x))$

9.2 (DAG-based DP for T_E). Apply the DAG-based decision procedure for T_E to the Σ_E -formulae of Exercise 9.1. Provide a level of detail as in Example 9.17.

9.3 (★Undecidable fragment). Show that allowing even one quantifier alternation (*i.e.*, $\exists x_1, \dots, x_k. \forall y_1, \dots, y_n. F[\bar{x}, \bar{y}]$) makes satisfiability in T_E undecidable.

9.4 (★DAG). Describe a data structure and algorithm for constructing the initial DAG in the congruence closure procedure. It should run in time approximately linear in the size of the formula.

9.5 (★PL & DAGs). This problem explores a concise representation of propositional logic formulae.

- (a) Describe a DAG-based representation of PL formulae.
- (b) If you have not already done so, consider that the logical connectives $\wedge$ and $\vee$ are associative and commutative. Improve your representation to exploit this observation.
- (c) Modify the CNF conversion algorithm of Section 1.7.3 to operate on your DAG-based representation. How large is the resulting CNF formula relative to the DAG?

9.6 (DP for T_{cons}). Apply the decision procedure for T_{cons} to the following T_{cons} -formulae. Provide a level of detail as in Example 9.20.

- (a) $\text{car}(x) = y \wedge \text{cdr}(x) = z \wedge x \neq \text{cons}(y, z)$
- (b) $\neg \text{atom}(x) \wedge \text{car}(x) = y \wedge \text{cdr}(x) = z \wedge x \neq \text{cons}(y, z)$

9.7 (Flawed DP for T_{cons}). Consider a variant of the T_{cons} -satisfiability procedure in which Steps 2 and 3 are swapped.¹ What is wrong with reversing these two steps? Identify a counterexample to its correctness: find a T_{cons} -unsatisfiable (conjunctive, quantifier-free) Σ_{cons} -formula that the incorrect procedure claims is satisfiable, and show the final DAGs for both the incorrect and the correct procedures.

¹ Suggested by a typo in [66].

9.8 (DP for quantifier-free T_A). Apply the decision procedure for quantifier-free T_A to the following Σ_A -formulae.

- (a) $a\langle i \triangleleft e \rangle[j] = e \wedge i \neq j$
- (b) $a\langle i \triangleleft e \rangle[j] = e \wedge a[j] \neq e$
- (c) $a\langle i \triangleleft e \rangle[j] = e \wedge i \neq j \wedge a[j] \neq e$
- (d) $a\langle i \triangleleft e \rangle\langle j \triangleleft f \rangle[k] = g \wedge j \neq k \wedge i = j \wedge a[k] \neq g$
- (e) $i_1 = j \wedge a[j] = v_1 \wedge a\langle i_1 \triangleleft v_1 \rangle\langle i_2 \triangleleft v_2 \rangle[j] \neq a[j]$